

Thresholding, Binarization, Blob analysis

The purpose of *image binarization* or *thresholding* is to separate pixels as belonging to 2 categories: OBJECTS (set to 1 or 255, WHITE) and BACKGROUND (set to 0, BLACK) by applying a threshold on the grayscales.

1) Create a program to read and visualize a grayscale image and its histogram, binarize it by choosing a global threshold manually. Display the result of the binarized image (show the threshold value in the title) and its histogram. Vary the threshold value and observe the result. Try on multiple images (**rice.png**, **coins.png**,...). When studying the histogram of the **ic.tif** image, how do you choose a good threshold value?

2) Test the **cv.threshold** and **cv.adaptiveThreshold** thresholding methods. How does adaptive thresholding work? What is its interest? What influence does the size of the neighborhood have on the result?

3) Apply an automatic **Otsu** thresholding and visualize the result. How does this method work?

4) Compare the results given by these methods on the same image **rice.png**.

See: https://docs.opencv.org/4.x/d7/d4d/tutorial_py_thresholding.html

Labeling, Blob Analysis

After the binarization of the image, the next step is often the analysis of each of the detected objects. To identify each object, *labeling into connected components* is performed to group together the adjacent neighboring pixels that do not belong to the background and form an object.

5) Test the **connectedComponents** method of the OpenCV library on the *formes.png* image and then on the binarized *rice.png* image. What values does it return?

6) Then test **connectedComponentsWithStats**. What additional information does it provide? View results.

7) On a particular image, highlight the variations of some parameters induced by the variation of the binarization threshold. For example, it will be possible to plot the histogram of the distribution of the areas of the related components and to calculate the mean and median area.